

// Jivops

Kibana

Guía completa — 20 lecciones

Exploración de datos · Visualización y dashboards · Alertas y observabilidad
· Administración y producción

Contenido

Módulo 1: Fundamentos y exploración de datos

- Lección 01 — Qué es Kibana y su relación con Elasticsearch
- Lección 02 — Data views — a qué datos se conecta Kibana
- Lección 03 — Discover — explorar y buscar documentos
- Lección 04 — KQL y filtros
- Lección 05 — Dev Tools — consola, Search Profiler y Grok Debugger

Módulo 2: Visualización y dashboards

- Lección 06 — Lens — visualizaciones con métricas y agrupaciones
- Lección 07 — Dashboards — paneles, controles, drilldowns y objetos guardados
- Lección 08 — Agregaciones — precisión, cardinalidad y límite de buckets
- Lección 09 — ES|QL — consultas en tubería
- Lección 10 — Rendimiento de dashboards

Módulo 3: Alertas, observabilidad y agentes

- Lección 11 — Reglas de alerta y acciones
- Lección 12 — Conectores y claves de cifrado
- Lección 13 — Observabilidad — logs, APM y correlación con ECS
- Lección 14 — Fleet y Elastic Agent
- Lección 15 — Elastic Security — reglas de detección y casos

Módulo 4: Administración y producción

- Lección 16 — Spaces — organizar objetos por equipo
- Lección 17 — Seguridad — autenticación, roles y TLS
- Lección 18 — Configuración y alta disponibilidad
- Lección 19 — Monitoreo, troubleshooting y actualizaciones
- Lección 20 — Proyecto final: Kibana listo para producción

Módulo 1: Fundamentos y exploración de datos

Lección 01 — Qué es Kibana y su relación con Elasticsearch

Es la interfaz web para explorar y visualizar los datos de Elasticsearch y administrar parte del stack; no almacena los datos de negocio, que viven en Elasticsearch, y guarda sus propios objetos (dashboards, vistas de datos, reglas) en índices del sistema de Elasticsearch, por lo que debe ser de una versión compatible con él.

Los dashboards y objetos se guardan en Elasticsearch y no en el servidor de Kibana, así que cualquier instancia lee el mismo estado; Kibana es un servicio sin estado propio.

Lección 02 — Data views — a qué datos se conecta Kibana

Indica sobre qué índices, alias o data streams se consulta y qué campo de tiempo se usa para el selector temporal; permite definir campos de runtime y formatos de visualización, y un patrón demasiado amplio mezcla datos distintos y hace las consultas más lentas.

La consulta se ejecuta sobre muchos índices que no hacen falta y consume más recursos, y mezcla campos de datos distintos, lo que confunde los resultados; un patrón acotado mejora velocidad y precisión.

Lección 03 — Discover — explorar y buscar documentos

Se combinan el selector de tiempo, la barra de consulta (KQL o Lucene), los filtros y los campos visibles; el histograma muestra cuándo ocurren los eventos, se pueden guardar búsquedas y reutilizarlas en dashboards, y conviene acotar el rango de tiempo para consultar menos datos.

Elasticsearch solo examina los documentos dentro del rango, y un rango amplio obliga a procesar muchos más datos; menos datos significa respuestas más rápidas y menos carga en el cluster.

Lección 04 — KQL y filtros

KQL permite consultas como campo:valor, combinadas con and, or y not, comodines, rangos y frases entre comillas; los filtros (pills) se aplican en contexto de filtro, sin cálculo de relevancia y con posibilidad de caché, y se pueden fijar entre aplicaciones; Lucene sigue disponible para sintaxis más avanzada.

Los filtros no calculan puntuación de relevancia y pueden almacenarse en caché, evitando trabajo cuando solo se necesita incluir o excluir documentos; las condiciones exactas como estado o servicio encajan bien como filtros.

Lección 05 — Dev Tools — consola, Search Profiler y Grok Debugger

La consola envía peticiones REST a Elasticsearch con los permisos del usuario conectado: sirve para probar consultas, ver mappings o consultar la salud del cluster; también incluye Search Profiler y Grok Debugger; como puede ejecutar DELETE o POST reales, hay que usarla con cuidado en producción.

La petición se ejecuta de verdad y puede borrar índices o documentos sin confirmación adicional; un error al escribir la ruta puede afectar a muchos datos, así que conviene usar permisos mínimos y comprobar la petición antes de enviarla.

Módulo 2: Visualización y dashboards

Lección 06 — Lens — visualizaciones con métricas y agrupaciones

Arrastrando campos: se combinan métricas (conteo, promedio, percentiles, cardinalidad) con agrupaciones como histograma de fechas, términos (top N) o filtros; sugiere el tipo de gráfico, y conviene recordar que los términos muestran los N valores principales y el resto se agrupa como otros.

Un campo con muchísimos valores distintos produciría miles de categorías ilegibles y costosas; limitar el número reduce el trabajo del cluster y mantiene el gráfico claro, y el resto de valores puede agruparse como otros.

Lección 07 — Dashboards — paneles, controles, drilldowns y objetos guardados

Un dashboard combina paneles que comparten selector de tiempo y filtros; se pueden añadir controles interactivos, fijar filtros, enlazar a otros dashboards (drilldowns) y compartir por URL; los objetos guardados se exportan e importan en formato NDJSON para moverlos entre entornos o guardarlos en control de versiones.

El enlace solo apunta al dashboard y cada panel consulta Elasticsearch con los permisos del usuario; sin permisos sobre los índices, los paneles aparecen vacíos o con error, porque compartir el objeto no concede acceso a los datos.

Lección 08 — Agregaciones — precisión, cardinalidad y límite de buckets

Algunas son aproximadas: los términos pueden tener error en campos de alta cardinalidad al calcular el top N por shards, y la cardinalidad usa un algoritmo aproximado (HyperLogLog++); además, un histograma con intervalo demasiado fino puede superar el límite de buckets (search.max_buckets), así que se ajusta el intervalo al rango de tiempo.

Genera un número enorme de buckets que supera el límite permitido y consume mucha memoria y tiempo en el cluster; conviene ajustar el intervalo al rango de tiempo consultado.

Lección 09 — ES|QL — consultas en tubería

Un lenguaje de consultas en tubería (FROM índice | WHERE ... | STATS ... BY ... | SORT ... | LIMIT ...) disponible en Discover y otras partes de Kibana en versiones recientes; cada paso transforma la tabla de resultados, lo que facilita explorar y agregar datos sin escribir Query DSL.

Cada paso se lee de arriba abajo como una transformación sobre el resultado anterior, el orden de las operaciones es explícito y más fácil de seguir, y se evita la anidación profunda de objetos.

Lección 10 — Rendimiento de dashboards

Suele deberse a demasiados paneles, rangos de tiempo muy amplios, agrupaciones sobre campos de alta cardinalidad o patrones de índices muy anchos; se mejora acotando tiempo y datos, usando filtros, reduciendo paneles, ajustando el intervalo de refresco y usando datos agregados o reducidos (downsampling) para históricos; el panel Inspect muestra tiempos y consultas.

Cada refresco lanza de nuevo todas las consultas de todos los paneles, y con muchos usuarios el número de consultas se multiplica; un intervalo razonable reduce la carga.

Módulo 3: Alertas, observabilidad y agentes

Lección 11 — Reglas de alerta y acciones

Una regla comprueba una condición cada cierto intervalo (por ejemplo, una consulta con umbral) mediante el Task Manager de Kibana y, al cambiar el estado de la alerta, ejecuta acciones a través de conectores; configurar que se notifique solo en cambios de estado y no en cada comprobación evita el ruido, y las reglas se ejecutan con una API key asociada a quien las creó o modificó.

Evita repetir el mismo aviso en cada comprobación mientras el problema continúa; el equipo recibe un aviso al empezar y otro al recuperarse, y con menos ruido las alertas importantes se atienden.

Lección 12 — Conectores y claves de cifrado

Los conectores (email, Slack, webhook, PagerDuty, Jira...) guardan credenciales como objetos cifrados con `xpack.encryptedSavedObjects.encryptionKey`; esa clave debe configurarse de forma persistente y ser la misma en todas las instancias de Kibana, o no se podrán descifrar los secretos tras un reinicio o desde otra instancia; conviene probar cada conector antes de usarlo en una regla.

Los objetos cifrados se guardan en Elasticsearch y cualquier instancia debe poder descifrarlos; con claves distintas una instancia no puede leer lo que cifró otra, y una clave persistente evita perder secretos tras reinicios.

Lección 13 — Observabilidad — logs, APM y correlación con ECS

Con Observability se exploran logs, métricas y trazas en un mismo sitio: APM muestra servicios, latencias y mapa de dependencias, y campos comunes como `@timestamp`, `service.name` o `trace.id` (ECS) permiten saltar de un error en un log a la traza y al servicio afectado; sin campos consistentes, esa correlación no funciona.

Los mismos nombres de campo permiten consultar y correlacionar datos de distintas fuentes, un dashboard o regla sirve para muchos servicios sin adaptarlo, y se salta de logs a trazas usando identificadores comunes.

Lección 14 — Fleet y Elastic Agent

Gestiona de forma central los Elastic Agents: se crean políticas de agente con integraciones (nginx, system, Kubernetes...) y los datos llegan a data streams con nombres como `logs-nginx.access-default`; permite actualizar agentes y cambiar la configuración desde la interfaz sin editar archivos en cada servidor.

Un cambio de política se aplica a todos los agentes asignados sin entrar servidor por servidor, se actualizan versiones y configuración desde un solo lugar, y se reduce el riesgo de configuraciones inconsistentes.

Lección 15 — Elastic Security — reglas de detección y casos

Usa reglas de detección (consultas KQL, EQL, umbrales o aprendizaje automático) que generan alertas sobre eventos de seguridad; las alertas se investigan con Timelines y se agrupan en casos; funciona mejor con datos normalizados en ECS, y las reglas necesitan ajuste para reducir falsos positivos.

Cada entorno tiene actividad normal distinta y una regla genérica puede disparar falsos positivos; ajustar excepciones reduce el ruido y deja visibles las alertas reales, porque un exceso de alertas inútiles hace que se ignoren las importantes.

Módulo 4: Administración y producción

Lección 16 — Spaces — organizar objetos por equipo

Separan dashboards, visualizaciones y otros objetos guardados por equipo o proyecto, y los roles definen a qué espacios y funciones accede cada usuario; sin embargo, no aíslan los datos de Elasticsearch: el acceso a los índices lo controlan los privilegios de índice del rol.

Los espacios organizan objetos de Kibana, pero los datos viven en Elasticsearch con sus propios permisos; un usuario con privilegios de índice podría consultarlos por otras vías, así que la restricción real de datos se hace con roles e índices.

Lección 17 — Seguridad — autenticación, roles y TLS

Un usuario necesita privilegios de Kibana (qué funciones y espacios) y de Elasticsearch (qué índices puede leer); la autenticación puede ser nativa, LDAP, SAML u OIDC con mapeo de roles, Kibana se conecta con un token de cuenta de servicio, y se usa TLS entre navegador, Kibana y Elasticsearch; las claves de cifrado de sesión, objetos y reportes deben configurarse.

Los privilegios de Kibana permiten usar la aplicación y los de Elasticsearch permiten leer los índices; sin los de índice los paneles se abren pero no muestran datos, porque son dos capas de autorización distintas.

Lección 18 — Configuración y alta disponibilidad

Kibana es un servicio sin estado propio, porque su estado vive en Elasticsearch: se pueden ejecutar varias instancias detrás de un balanceador, con la misma configuración de claves de cifrado y varios hosts de Elasticsearch en kibana.yml; el Task Manager reparte la ejecución de reglas entre instancias, y conviene indicar la URL pública con server.publicBaseUrl.

Si una instancia cae, el balanceador envía el tráfico a las demás; como el estado está en Elasticsearch, todas sirven los mismos objetos, y se puede actualizar o reiniciar una instancia sin cortar el servicio.

Lección 19 — Monitoreo, troubleshooting y actualizaciones

Se comprueba su salud con la API de estado (/api/status) y se monitoriza con Stack Monitoring o métricas externas (memoria, retardo del bucle de eventos, tiempos de respuesta); para dashboards lentos se usa Inspect y los slow logs de Elasticsearch, y antes de actualizar se hace una copia del estado (snapshot de los índices de Kibana) y se revisan los avisos del asistente de actualización.

La actualización migra los objetos guardados y un fallo podría dejarlos dañados; una copia permite volver atrás sin perder dashboards ni reglas y reduce el riesgo de la actualización.

Lección 20 — Proyecto final: Kibana listo para producción

Data views acotadas y KQL o ES|QL bien usados, dashboards ligeros contruidos con Lens, reglas con conectores y notificación solo en cambios de estado, Spaces con roles de mínimo privilegio y TLS, varias instancias con claves de cifrado compartidas, y monitoreo y copias de los objetos guardados antes de actualizar.

Intenta mencionar al menos dos pilares: data views acotadas y dashboards ligeros con Lens, reglas con conectores y notificación en cambios de estado, Spaces con roles de mínimo privilegio y TLS, varias instancias con claves compartidas, o monitoreo y copias antes de actualizar.